Which one you use is largely a matter of preference.

7.5.2. filter

Suppose you have an array of your friends' addresses, and you want to know which ones live in `'CoffeeVille'`:

```
friends = [
  { name: 'bob', location: 'CoffeeVille' },
  { name: 'tom', location: 'JavaLand' },
  { name: 'sam', location: 'PythonTown' },
  { name: 'jenny', location: 'RubyCity' }
]
```

You can find out with a filter:

```
friends.filter (friend) -> friend.location is 'CoffeeVille'
```

This looks similar when expressed using a comprehension:

```
friend for friend in friends when friend.location is 'CoffeeVille'
```

There's no clear-cut winner. Does this means that comprehensions are overrated? Suppose you have an array of your friends in a variable named `mine`:

```
mine = ['Greg Machine', 'Bronwyn Peters', 'Sylvia Rogers']
```

Consider this to be your set of friends. Now, suppose you spark up a conversation at a party, and you want to know if you have any friends in common with the person you're talking to. Consider the set of their friends to be named `yours`; the solution with a comprehension is elegant:

```
common = (friend for friend in mine when friend in yours)
```

It's no coincidence that this set relationship is elegantly expressed using a comprehension because the syntax for comprehensions is based on a mathematical notation for describing sets. So if you're dealing with sets, comprehensions are a natural fit.